林明康(111012062)醫環26 Deep Learning in Biomedical Optics Imaging HW3

Data (數量):
Train_set: Normal (1000), Pheumonia (1000)
Test_set: Normal (234), Pheumonia (390)
Validation_set: Normal (8), Pneumonia (8)
Optimizer: Adam (learning_rate = 0.001, weight_decay = 0.001)

基於這個初始設定, 分為三個階段:

1. 模型架構探索:系統性地比較不同模型架構的效能, 從基礎的 MLP 模型, 進一步導入了更能捕捉圖像空間特徵的卷積神經網路(CNN), 而後採用如 ResNet18 與 EfficientNet-B0 等預訓練模型, 以探索遷移學習的潛力。
2. Hyperparameter Tuning:在比較了不同模型的基礎表現後, 我們轉向對訓練過程進行微調。這包括將優化器從 Adam 更換為更穩健的 AdamW, 並對學習率等關鍵超參數進行調整, 旨在提升訓練的穩定性與收斂效果 。
3. 數據策略優化:在初步實驗中, 觀察到極小的驗證集導致了評估指標的劇烈震盪, 無法可靠地指導模型優化 。因此, 最終的優化策略是從根本上解決此問題, 透過重新劃分數據集來建立一個更大、更具統計代表性的驗證集 。結合前兩個階段得出的最佳模型(ResNet18)與微調後的hyperparameter, 進行了最後的訓練, 以期獲得一個近乎完美的性能表現

調整方向:

1. 數據集劃分 (Train/Validation Split)

- 將原本僅有 16 張圖片的驗證集, 改為從 2000 張訓練集中切分出 416 張作為新的驗證集。

2. 損失函數 (Loss Function): nn.BCEWithLogitsLoss()

- 數值穩定性:BCEWithLogitsLoss 將 Sigmoid 函數和二元交叉熵的計算合併在一步完成。這種做法在數學上可以避免極端值(接近 0 或 1)造成的浮點數運算不穩定問題, 讓訓練過程更穩健。

3. 優化器 (Optimizer): AdamW

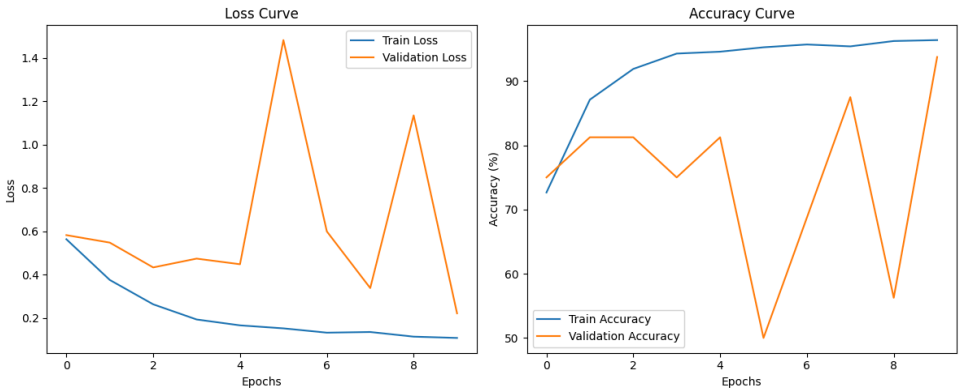
- AdamW 是 Adam 的一個改良版, 它們的主要區別在於處理權重衰減 (Weight Decay) 的方式。Adam的權重衰減與梯度更新是耦合在一起的, 有時候效果不是最優的。AdamW則是將權重衰減的步驟與梯度更新解耦, 使其更接近原始論文中的定義。在實踐中, AdamW通常能提供更好的泛化能力, 幫助模型更好地防止過擬合。對於微調(fine-tuning)預訓練模型來說, AdamW 常常是首選。

4. 學習率 (Learning Rate): 0.000

5. 早停機制 (Early Stopping)

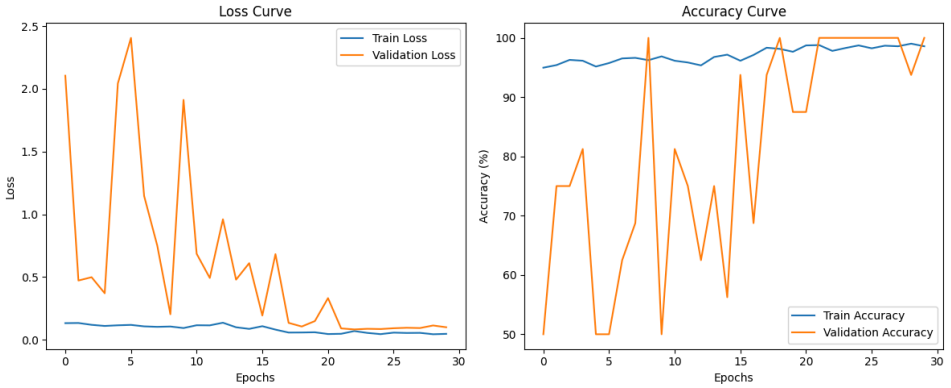
1. Trial 1: MLP Model (GPU: Apple Silicon GPU-mps)
   (0): Flatten(start_dim=1, end_dim=-1)
   (1): Linear(in_features=65536, out_features=64, bias=True)
   (2): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
   (3): ReLU()
   (4): Dropout(p=0.5, inplace=False)
   (5): Linear(in_features=64, out_features=64, bias=True)
   (6): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
   (7): ReLU()
   (8): Dropout(p=0.5, inplace=False)
   (9): Linear(in_features=64, out_features=64, bias=True)
   (10): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
   (11): ReLU()
   (12): Dropout(p=0.5, inplace=False)
   (13): Linear(in_features=64, out_features=1, bias=True)
   (14): Sigmoid()

   a. Epochs: 10



   Test Accuracy: 80.89%
   Test Loss: 0.4977

   b. Epochs: 30

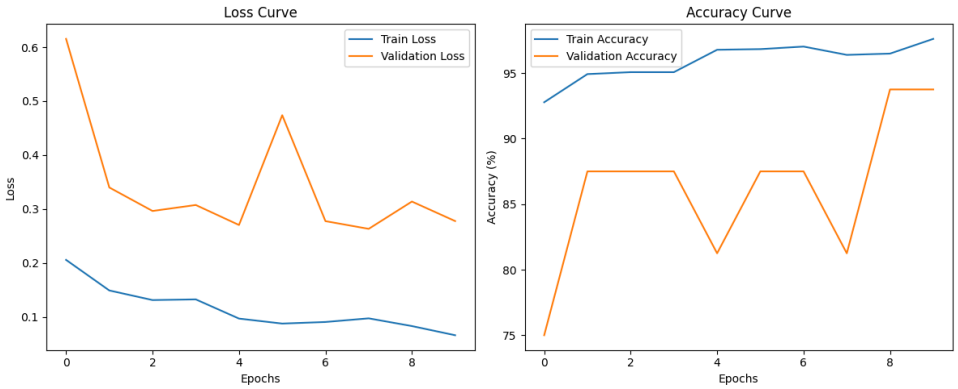


   Test Accuracy: 80.68%
   Test Loss: 0.4691

2. Trial 2: 簡易版 CNN Model (GPU: NVIDIA Tesla T4)

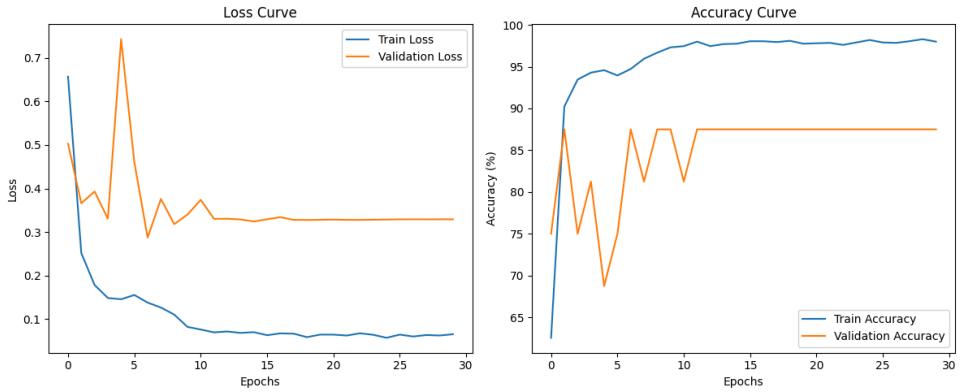
(0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(1): ReLU()
(2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(3): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(4): ReLU()
(5): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(6): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
(7): ReLU()
(8): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
(9): Flatten(start_dim=1, end_dim=-1)
(10): Linear(in_features=65536, out_features=128, bias=True)
(11): ReLU()
(12): Dropout(p=0.5, inplace=False)
(13): Linear(in_features=128, out_features=1, bias=True)
(14): Sigmoid()

a. Epochs: 10



Test Accuracy: 81.82%
Test Loss: 0.5981

b. Epochs: 30

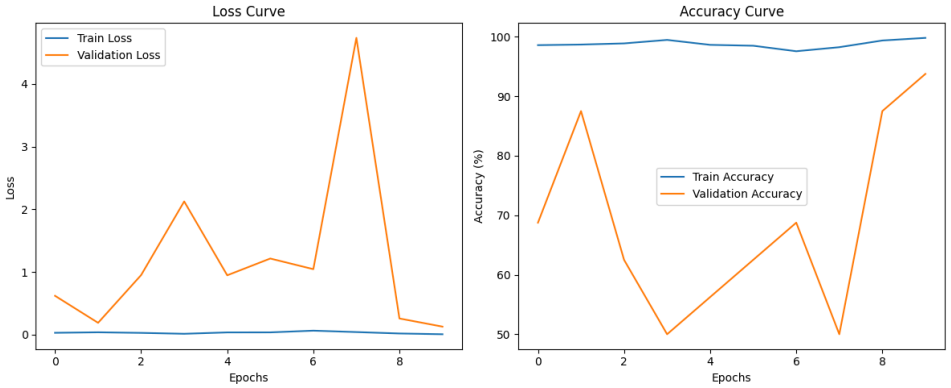


Test Accuracy: 85.21%
Test Loss: 0.3490

3. Trial 3: ResNet18 Model (GPU: NVIDIA GeForce RTX 3050 Laptop GPU)

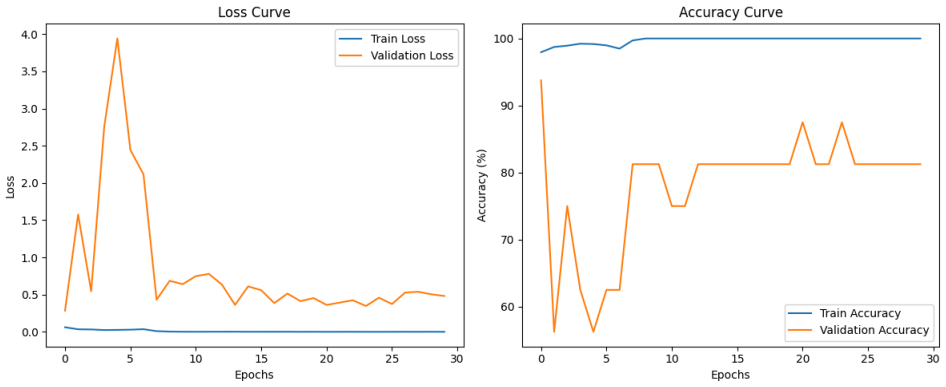
```
from torchvision.models import resnet18, ResNet18_Weights
model = resnet18(weights=ResNet18_Weights.DEFAULT)
model.conv1 = nn.Conv2d(1, 64, kernel_size=(7, 7), stride=(2, 2),
padding=(3, 3), bias=False)
num_ftrs = model.fc.in_features
model.fc = nn.Sequential(
  nn.Linear(num_ftrs, 1),
  nn.Sigmoid()
```

a. Epochs : 10



Test Accuracy: 81.09%
Test Loss: 0.7636

b. Epochs : 30

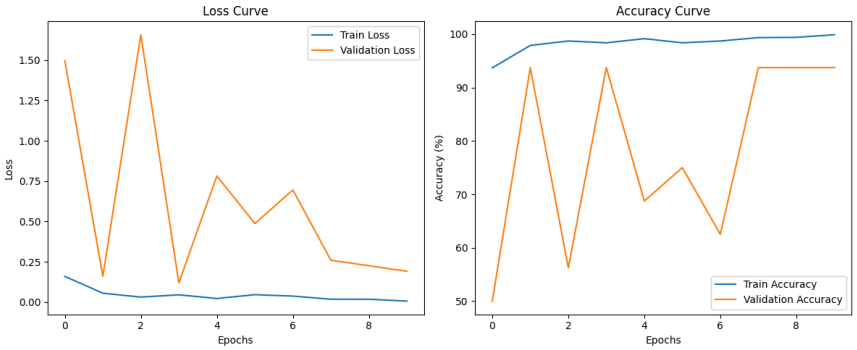


Test Accuracy: 84.53%
Test Loss: 0.3889

4. Trial 4: EfficientNet_B0 Model (GPU: NVIDIA GeForce RTX 3050 Laptop GPU)

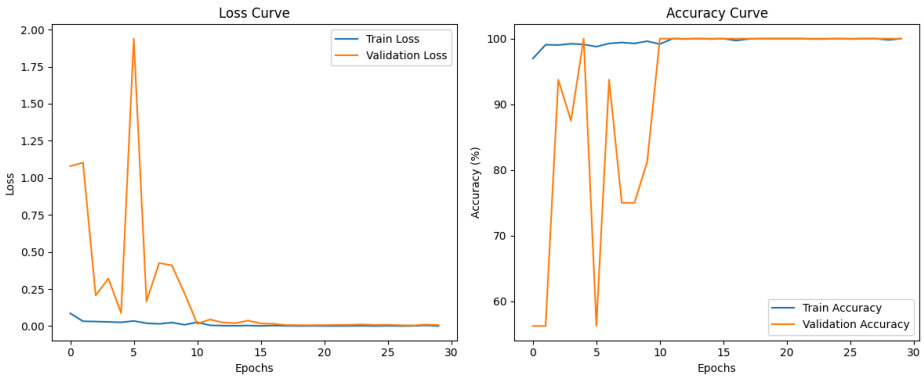
```
from torchvision.models import efficientnet_b0, EfficientNet_B0_weights
model = efficientnet_b0(weights=EfficientNet_B0_weights,default)
original_conv = model.features[0][0]
model.features[0][0] = nn.Conv2d(1,
                                 original_conv.out_channels,
                        kernel_size=original_conv.kernel_size,
                                  stride=original_conv.stride,
                                 padding=original_conv.padding,
                                   bias=False)
num_ftrs = model.classifier[1].in_features
model.classifier[1] = nn.Sequential(
    nn.Linear(num_ftrs, 1),
    nn.Sigmoid()
```

a. Epochs: 10



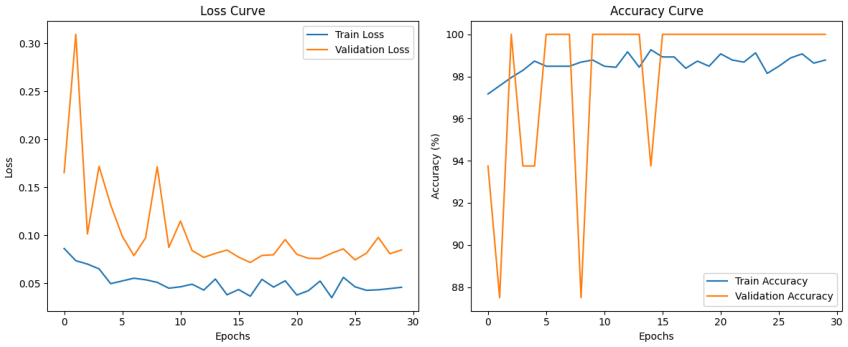
Test Accuracy: 84.79%
Test Loss: 0.3792

b. Epochs: 30



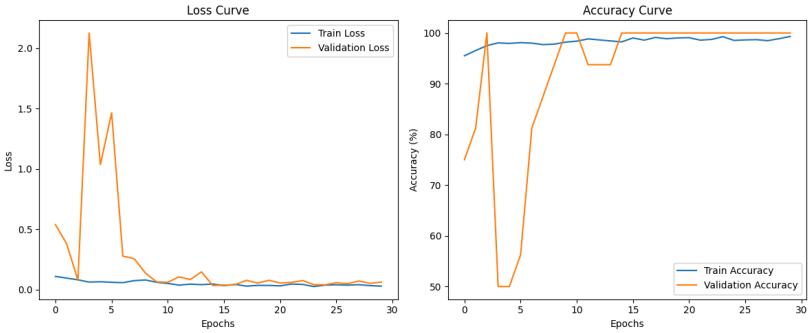
Test Accuracy: 80.78%
Test Loss: 0.8025

Another Trial Set:
Optimizer: AdamW (Learning_rate = 0.0001)
Model: MLP Model (GPU: Apple Silicon GPU-mps)



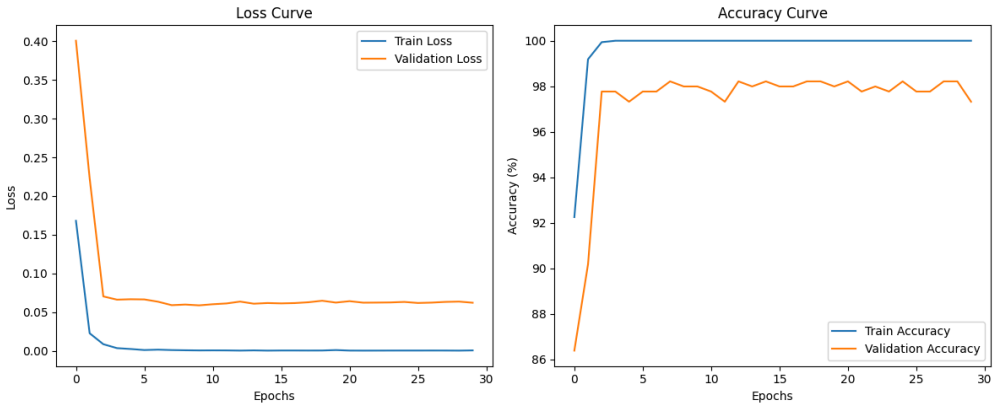
Test Accuracy: 82.24%
Test Loss: 0.5083

Optimizer: AdamW (Learning_rate = 0.001)



Test Accuracy: 83.33%
Test Loss: 0.4810

Optimizer: AdamW (Learning_rate = 0.0001) – Epochs: 30
Model: ResNet18 (GPU: NVIDIA GeForce RTX 3050 Laptop GPU)
Data: Train(800/800), Validation(208/208), Test (234/390)
Criterion: nn.BCEWithLogitsLoss()



Test Accuracy: 87.92%
Test Loss: 0.5306

Based on a comprehensive series of trials, the optimal solution for this pneumonia classification task was achieved by fine-tuning a ResNet18 model under an improved data and training strategy. The initial experiments, which evaluated MLP, custom CNN, ResNet18, and EfficientNet-B0 models, were fundamentally limited by an extremely small validation set of only 16 images, resulting in highly unstable and unreliable validation metrics across all models. Recognizing this limitation, a final optimized experiment was conducted by re-partitioning the data to create a robust validation set of 416 images , switching to the AdamW optimizer with a lower learning rate of 0.0001 , and using BCEWithLogitsLoss. This refined approach stabilized the training process and allowed the ResNet18 model to reach a peak test accuracy of 87.92%, demonstrating that a sound validation strategy and careful hyperparameter tuning are critical to achieving superior performance.